

KẾ HOẠCH TRIỂN KHAI AN TOÀN TUYỆT ĐỐI (GIT WORKFLOW)

Tài liệu này hướng dẫn chi tiết quy trình Git Workflow cho cả nhóm và lộ trình triển khai chi tiết các Phase tiếp theo để đảm bảo dự án báo cáo Manga Workflow diễn ra an toàn, không bị lỗi cú pháp LaTeX và không bị mất code.



1. QUY TRÌNH GIT WORKFLOW CHO CẢ NHÓM

Mục tiêu: Đóng băng nhánh `main` và nhánh tích hợp chung `restructure-toc-final`. Toàn bộ thành viên (kể cả Leader) đều làm việc trên nhánh riêng.



Sơ đồ luồng Git (Git Branching Model)

`main` (Stable - Luôn biên dịch chạy được)



└─ `restructure-toc-final` (Nhánh tích hợp chung của cả nhóm)



└─ `feature/tan-lead` (Leader Trương Trọng Tấn)

└─ `feature/han-srs-func1` (Giang Thị Ngọc Hân)

└─ `feature/thao-srs-func2` (Nguyễn Thanh Thảo)

└─ `feature/ngan-testing` (Dương Kim Ngân)

└─ `feature/trucngan-sdd` (Nguyễn Thị Trúc Ngân)

└─ `feature/hanh-sdd-ui` (Phan Thị Hạnh)



2. QUY TRÌNH LÀM VIỆC DÀNH CHO THÀNH VIÊN (Kể cả Leader)

Mỗi khi bắt đầu viết một nội dung mới, hãy thực hiện theo đúng các bước sau:

Bước 1: Cập nhật nhánh tích hợp chung ở máy cá nhân

Trước khi tạo nhánh mới, phải lấy code mới nhất về máy:

```
git checkout restructure-toc-final
git pull origin restructure-toc-final
```

Bước 2: Tạo nhánh con riêng để làm việc

Đặt tên nhánh theo đúng quy ước `feature/ten-chuc-nang`: * Ví dụ cho Tấn (Leader): `git checkout -b feature/tan-lead` * Ví dụ cho Hân: `git checkout -b feature/han-srs-func1`

```
git checkout -b feature/your-name-task
```

Bước 3: Thực hiện viết báo cáo và lưu lại

Khi bạn hoàn thành một phần nhỏ (ví dụ: xong 1 use case hoặc xong 1 mục):

```
git add .
git commit -m "Mô tả chi tiết nội dung đã viết bằng tiếng Việt"
```

Bước 4: Đẩy nhánh con lên GitHub

```
git push origin feature/your-name-task
```

Bước 5: Tạo Pull Request (PR) trên GitHub

1. Truy cập vào trang GitHub của dự án.
 2. Nhấp vào nút **"Compare & pull request"** màu vàng của nhánh bạn vừa đẩy lên.
 3. Chọn nhánh đích cần merge vào là `restructure-toc-final` (Không được chọn `main`).
 4. Viết mô tả ngắn gọn những mục đã hoàn thành rồi bấm **Create pull request**.
-

3. QUY TRÌNH DUYỆT BÀI & TỔNG HỢP DÀNH CHO LEADER (Trương Trọng Tấn)

Với vai trò Leader, bạn sẽ là người kiểm soát chất lượng cuối cùng trên nhánh `restructure-toc-final`.

Bước 1: Xem xét Pull Request trên GitHub

Khi một thành viên tạo PR gửi vào `restructure-toc-final`, bạn cần kiểm tra: 1. Các file thay đổi có đúng với phân công hay không (có bị ghi đè nhầm file của người khác không). 2. Kiểm tra nhanh xem có đoạn code LaTeX nào bị lỗi cú pháp hiển thị hay không.

Bước 2: Kiểm tra biên dịch (Compile) cục bộ trước khi Merge (Quan trọng!)

Để đảm bảo code của thành viên không làm hỏng cả báo cáo, bạn hãy kéo thử code của họ về máy và build thử:

```
# Lấy danh sách nhánh mới nhất từ GitHub
git fetch origin

# Chuyển sang nhánh của thành viên để test thử
git checkout feature/ten-nhanh-cua-thanh-vien
git pull origin feature/ten-nhanh-cua-thanh-vien

# Chạy thử lệnh biên dịch LaTeX
pdflatex -interaction=nonstopmode main.tex
```

- Nếu file PDF xuất ra thành công và không báo lỗi `Fatal error` -> Code an toàn.
- Nếu bị lỗi biên dịch -> Yêu cầu thành viên sửa lại trên nhánh của họ trước khi duyệt.

Bước 3: Chấp nhận Merge trên GitHub

Nếu mọi thứ đều hoạt động tốt, bạn nhấn nút **"Merge pull request"** trên trang GitHub để gộp code vào `restructure-toc-final`.

⚡ 4. CÁCH XỬ LÝ XUNG ĐỘT (GIT CONFLICT) CHO LEADER

Nếu hai người vô tình chỉnh sửa cùng một dòng trong file cấu hình (như `main.tex` hoặc `sections/thuat_ngu.tex`), GitHub sẽ báo **"This branch has conflicts that must be resolved"**.

Các bước xử lý conflict an toàn:

1. Chuyển về nhánh chung và cập nhật code mới nhất: `bash git checkout restructure-toc-final git pull origin restructure-toc-final`
2. Chuyển sang nhánh con của bạn (hoặc nhánh đang bị conflict): `bash git checkout feature/ten-nhanh-bi-conflict`
3. Gộp nhánh `restructure-toc-final` vào nhánh con: `bash git merge restructure-toc-final`
4. Git sẽ báo những file bị conflict. Mở các file đó lên bằng VS Code. Bạn sẽ thấy các ký tự: ````text <<<<<< HEAD` Đoạn code của bạn ở máy cá nhân `=====` Đoạn code mới nhất trên server do người khác vừa merge lên



restructure-toc-final ``

5. Chọn giữ lại đoạn code đúng nhất (hoặc gộp cả hai), xóa bỏ các dòng ký tự `<<<<<<`, `=====`, `>>>>>>`.
 6. Lưu file lại và commit để kết thúc xung đột: `bash git add . git commit -m "Fix: giải quyết xung đột với nhánh restructure-toc-final" git push origin feature/ten-nhanh-bi-conflict`
 7. Lúc này Pull Request trên GitHub sẽ tự động chuyển sang màu xanh và sẵn sàng để merge.
-



5. LỘ TRÌNH TRIỂN KHAI CHI TIẾT TRONG TƯƠNG LAI



PHASE 2: Di chuyển nội dung cũ (Thời gian: 2 ngày)

- **Mục tiêu:** Di chuyển an toàn toàn bộ nội dung từ các file `.tex` cũ sang các file `.tex` mới tương ứng theo bảng mapping.
- **Cách làm:** Các thành viên được phân công ở Phase 2 sẽ copy nội dung thô của các phần tương ứng, dán vào file mới, sửa lại nhãn `\label` và liên kết hình ảnh cho chuẩn xác.
- **Checkpoint:** Cuối Phase 2, Leader chạy biên dịch thử `main.tex` trên nhánh `restructure-toc-final`. Toàn bộ nội dung cũ phải hiển thị đúng trong mục lục mới, không mất mát thông tin.



PHASE 3: Viết mới các nội dung còn thiếu (Thời gian: 7 ngày)

- **Mục tiêu:** Viết mới hoàn toàn 43 mục còn thiếu (WBS, Use Case Spec, Mockup UI, Test Cases, Release Package).
- **Quy tắc:** Viết đến đâu, tạo test case và review chéo đến đó. Các thành viên liên tục cập nhật tiến độ vào file `checklist_chuan_hoa_srs.md`.
- **Checkpoint:** Biên dịch thành công bản báo cáo nháp đầy đủ nội dung.



PHASE 4: Đồng bộ định dạng & Nghiệm thu (Thời gian: 2 ngày)

- **Mục tiêu:** Sửa đổi toàn bộ lỗi chính tả, căn lề, font chữ, kích thước bảng và liên kết hình vẽ cho thật đồng đều và đẹp mắt.
- **Nghiệm thu:** Tạo Pull Request gộp `restructure-toc-final` vào `main` để chốt báo cáo chính thức nộp giảng viên.